

Product Requirements Document (PRD)

1. Cover Page

- **Project Name:** AegisX – AI-Powered Cybersecurity Assessment Platform
- **Document Type:** Product Requirements Document (PRD)
- **Version:** 1.0.0
- **Author:** 3rd-Year Computer Science Engineering Student
- **Target Audience:** Recruiters, Hackathon Judges, Open-Source Contributors, Technical Interviewers
- **Repository:** [GitHub Repository Link Placeholder]
- **Status:** In Development / Baseline Release

2. Version History

Version	Date	Author	Description of Changes
v1.0.0	Current	Engineering Lead	Initial PRD baseline covering core scanning, scoring engine, dashboard, and reporting features.

3. Executive Summary

AegisX is a lightweight, modular, and extensible cybersecurity assessment platform designed to automate initial reconnaissance, attack surface analysis, and vulnerability scoring for web applications and network endpoints. By consolidating port scanning, service identification, SSL/TLS analysis, and security header checks into an intuitive Streamlit dashboard, AegisX lowers the barrier to entry for security auditing. It equips developers and security enthusiasts with actionable remediations, dynamic risk scores, and downloadable executive reports.

4. Project Vision

To build an open-source, lightweight security engine that simplifies attack surface visibility, bridging the gap between raw command-line security tools and actionable security posture management.

5. Problem Statement

Small-to-medium enterprises (SMEs), developers, and security enthusiasts often lack access to fast, visual, and unified security auditing tools. Existing enterprise solutions (e.g., Nessus, Qualys) are complex and expensive, while basic CLI utilities (e.g., `nmap`, `dig`, `curl`) require manual correlation of disparate output formats. There is a strong need for an integrated tool that executes multi-layered reconnaissance, correlates findings into a singular risk score, and provides instant remediation guidance.

6. Business Objectives

- **Portfolio Demonstration:** Showcases full-stack software engineering, network protocol fundamentals, applied security principles, and clean UI design.
- **Open Source Utility:** Provides developers with a free tool to run pre-deployment posture checks on non-production assets.
- **Product Thinking:** Demonstrates end-to-end product owner capabilities, ranging from core functional definitions to roadmap planning.

7. Project Goals

- Achieve a target scan completion time of under 30 seconds for standard target profiles (Top 100 ports + web stack audit).
- Maintain a modular, decoupled Python architecture to enable easy additions of new scanning modules.
- Provide clear risk scores (\$0.0 - 10.0\$) using transparent, rule-based heuristic calculations.
- Export comprehensive executive summary reports in both PDF and CSV formats.

8. Scope

8.1 In-Scope (Phase 1 / MVP)

- Network Port Scanning (Top common ports).
- Banner Grabbing & Service Identification.
- HTTP Security Header Inspection.
- SSL/TLS Certificate Validity & Cipher Suite Checks.
- DNS Record Resolution & WHOIS Data Retrieval.
- Heuristic Risk Scoring Engine & Remediation Mapping.
- Streamlit Interactive Dashboard UI.
- PDF & CSV Audit Report Generation.

8.2 Out-of-Scope (Phase 1 / MVP)

- Active Exploitation or Automated Penetration Testing.
- Deep Web Vulnerability Fuzzing (e.g., SQLi/XSS active payloads).
- User Authentication, RBAC, and Multi-tenancy.
- Persistent Database Storage & Scan History (Deferred to Phase 2).
- Live AI API Integrations (Deferred to Phase 2).

9. User Personas

Persona A: Alex – The Junior Security Analyst

- **Goal:** Quickly perform passive/light-active external audits on assigned target domains without configuring dozens of CLI flags.
- **Pain Point:** Spending excess time manually formatting scan outputs into client-ready reports.

Persona B: Dev - The Full-Stack Engineer

- **Goal:** Verify HTTP headers, SSL validity, and exposed ports before pushing web apps to production.
- **Pain Point:** Lacks deep domain knowledge in cybersecurity configurations and needs straight-to-the-point remediation steps.

10. Functional Requirements

ID	Module	Requirement Description	Priority
FR-01	Port Scanner	The system shall scan targeted host ports using asynchronous socket connections.	High
FR-02	Banner Grabber	The system shall read response headers on open ports to deduce service software and version signatures.	Medium
FR-03	Web Auditor	The system shall evaluate HTTP response headers (e.g., HSTS, CSP, X-Frame-Options, X-Content-Type-Options) against security best practices.	High
FR-04	SSL/TLS Inspector	The system shall fetch server certificate details, verify expiration dates, issuer trust, and identify weak protocol versions.	High
FR-05	Reconnaissance	The system shall fetch DNS records (A, AAAA, MX, TXT, NS) and WHOIS domain ownership data.	Medium
FR-06	Risk Engine	The system shall calculate an overall posture score (\$0.0 - 10.0\$) by weighing open high-risk ports, missing headers, and SSL flaws.	High

FR-07	Dashboard	The system shall display scan summaries, metrics charts, tabbed findings, and risk indicators via Streamlit.	High
FR-08	Reporting	The system shall render printable PDF executive reports and downloadable CSV raw data dumps.	Medium

11. Non-Functional Requirements

- **Performance:** A standard scan (Top 100 ports + Web Audit) must complete in ≤ 30 seconds on standard broadband connections.
- **Usability:** Dashboard controls must be intuitive, requiring zero CLI commands from the user during runtime.
- **Security:** The system must strictly enforce input validation (IP/Domain sanitization) to prevent command injection vulnerabilities.
- **Portability:** The platform must execute seamlessly on Linux, macOS, and Windows via standard Python execution (`python -m streamlit run app.py`).
- **Maintainability:** Codebase must follow standard PEP 8 styling guidelines and use modular class structure for scanning engines.

12. User Stories

US-01: Quick Attack Surface Audit

- **As a** DevSecOps Engineer,
- **I want to** enter a target domain and execute a consolidated scan,
- **So that** I can identify exposed ports and missing security headers in a single click.

US-02: Executive Reporting

- **As a** Security Consultant,
- **I want to** export scan findings to a PDF,
- **So that** I can share an easy-to-read audit summary with non-technical stakeholders.

US-03: Actionable Remediation Guidance

- **As a** Junior Developer,
- **I want to** view contextual recommendations alongside detected vulnerabilities,
- **So that** I can fix misconfigurations without spending hours researching solutions.

13. Success Metrics

- **Scan Accuracy:** 100% correlation on basic header checks compared to standard `curl -I` outputs.
- **Scan Speed:** Average completion time ≤ 30 seconds.

- **Report Generation:** Zero crash rate when compiling PDF reports across valid target results.

14. Constraints

- **Legal Compliance:** Scans must be performed strictly on authorized target domains/IPs or local testing environments (e.g., `localhost`).
- **Network Limits:** Socket timeouts must be handled gracefully to prevent scan hangs caused by silent firewalls or dropped packets.
- **No Active Exploitation:** The tool is intentionally limited to passive/defensive scanning and basic probing; no exploit payloads will be sent.

15. Risks & Mitigations

Risk Description	Severity	Mitigation Strategy
Target IP Rate Limiting / Blocking	Medium	Implement socket timeouts, configurable thread limits, and user-agent rotation.
Command Injection via Malicious Input	High	Enforce strict regex validation for IP addresses and domain names before initiating network calls.
Excessive Execution Times	Medium	Utilize Python <code>asyncio</code> or concurrent threading for parallel port checking.

16. Assumptions

- The host machine executing AegisX has active internet connectivity.
- Target hosts respond to standard TCP connection requests unless restricted by upstream firewalls.
- Users are aware of legal restrictions surrounding unauthorized network port scanning.

17. Deliverables

1. **Source Code:** Fully documented Python repository organized into `core/`, `ui/`, and `reports/` packages.
2. **Documentation:** Detailed `README.md`, PRD documentation, and inline docstrings.
3. **UI Application:** Interactive Streamlit web interface.
4. **Sample Outputs:** Pre-generated sample PDF and CSV reports for test targets.

18. Milestones & Timeline

Plaintext

PROJECT TIMELINE		
Phase 1: Core Engine Development (Week 1)		
├──	Setup modular structure & regex validation	
└──	Build Port Scanner & Banner Grabber modules	
Phase 2: Web Security & Scoring Logic (Week 2)		
├──	Implement HTTP Header & SSL/TLS modules	
└──	Develop Risk Scoring Algorithm & Remediation Mapping	
Phase 3: UI Dashboard & Reporting (Week 3)		
├──	Build Streamlit UI layout & dynamic visualization	
└──	Integrate ReportLab PDF & CSV generation export	
Phase 4: Testing, Hardening & Documentation (Week 4)		
├──	Perform edge-case testing & input sanitization	
└──	Finalize GitHub README, PRD, and release build	

19. Tech Stack

- **Core Logic:** Python 3.10+
- **UI Framework:** Streamlit
- **Networking & Protocol Parsing:** `socket`, `ssl`, `requests`, `dnspython`, `python-whois`
- **Data Handling:** `pandas`
- **Visualization:** `plotly` / `altair`
- **PDF Generation:** `reportlab`

20. Future Roadmap (Post-MVP Expansion)

Phase 2: AI Enhancements & Database Integration

- **AI Security Explanations:** Integrate LLM APIs (e.g., OpenAI / Gemini) to auto-generate context-aware threat explanations for identified risks.
- **CVE Lookup Module:** Query public NIST NVD APIs to map grabbed service banners directly to known CVEs.
- **Persistence Layer:** Integrate SQLite / PostgreSQL with SQLAlchemy to store historical scan data and track security posture trends over time.

Phase 3: Platform Maturity

- **Threat Intelligence APIs:** Integrate AbuseIPDB, VirusTotal, and Shodan API queries.
- **Automated Scheduling:** Implement cron-based or background worker scheduled scans with email/webhook notifications.
- **Multi-User Auth:** Implement JWT-based authentication and role-based access control (RBAC).

21. Acceptance Criteria

Scenario 1: Successful Web & Network Assessment * **Given** an authorized domain target (e.g., [example.com](#)), * **When** the user inputs the domain and clicks "**Start Scan**", * **Then** the engine executes port checks, HTTP header checks, SSL checks, and DNS queries, returning a calculated Risk Score, organized dashboard metrics, and downloadable PDF/CSV reports in under 30 seconds.

Scenario 2: Invalid Input Handling * **Given** an invalid input string (e.g., [https://invalid_target; rm -rf /](#)), * **When** the user submits the input, * **Then** the input validator traps the input, halts execution, and displays an error message without breaking application state.